

# RECURSION

Date: 25/05/24

Topic: Recursion

## 1. What is Recursion?

- Recursion is a programming technique where a function calls itself to solve a smaller instance of the same problem.
- It breaks a problem into smaller subproblems of the same type.

## 2. Key Components

- Base Case: The condition that stops the recursion.
- Recursive Case: The part where the function calls itself with a smaller/simpler input.

## 3. General Structure

```
function f(input):  
    if base_condition:  
        return base_result    // base case  
    else:  
        return f(smaller_input) // recursive case
```

## 4. Examples

- Factorial ( $n!$ )

Base Case:  $n = 0 \Rightarrow 0! = 1$

Recursive Case:  $n > 0$

$$n! = n \times (n-1)!$$

- Fibonacci ( $F_n$ )

Base Cases:  $F_0 = 0, F_1 = 1$

Recursive Case:  $n > 1$

$$F_n = F_{n-1} + F_{n-2}$$

## 5. Pros and Cons

### Pros

- Easy to write and understand
- Matches problems with recursive structure
- Useful for tree and graph problems

### Cons

- Higher function call overhead
- Uses more memory (call stack)
- Can be inefficient without optimization (e.g., memoization)

## 6. Tip

- Always ensure a base case is reachable; otherwise, recursion leads to infinite calls (stack overflow).



# TWO POINTER APPROACH

## 1. What is it?

- A technique that uses two pointers (indices) to traverse a data structure (usually an array or string) to solve problems efficiently.
- Helps reduce time complexity, often from  $O(n^2)$  to  $O(n)$ .

## 2. When to use?

- The data is sorted.
- You need to check pairs or subarrays.
- You need to reverse, partition, or compare from both ends.
- Common in problems involving sums, duplicates, palindromes, etc.

## 3. Common Patterns

- ① **Opposite Direction:** Start from both ends and move towards the center.
- ② **Same Direction:** Start from the beginning and move both pointers forward.
- ③ **Sliding Window:** Maintain a window with two pointers (left, right).

## 4. Basic Template (Opposite Direction)

// Example: Array arr of size n

left = 0

right = n - 1

while left < right:

- Do something with arr[left] and arr[right]
- If condition met, move pointers
- else, move pointers based on the problem

### Pointer Moves:

- left += 1 → move right
- right -= 1 → move left
- left += 1 and right -= 1 → move both

## 5. Examples

- Two Sum in Sorted Array (find a pair with target sum)
- Remove Duplicates from Sorted Array (use slow & fast pointers)
- Valid Palindrome (check characters from both ends)
- 3Sum / 4Sum Problems (fix one element, two pointer for the rest)

### Remember:

Move pointers smartly based on the condition to avoid missing solutions.





# DYNAMIC PROGRAMMING

## 1. WHAT IS DYNAMIC PROGRAMMING?

- A method for solving complex problems by breaking them into overlapping subproblems.
- Solves each subproblem **once** and **stores** the result to avoid recomputation.
- Applicable when the problem has:
  - Optimal Substructure
  - Overlapping Subproblems

## 2. HOW IT WORKS

- ① Define the **state** (what subproblem is represented).
- ② Write the **recurrence relation**.
- ③ Determine the **base case(s)**.
- ④ Compute the answer using **memoization** (top-down) or **tabulation** (bottom-up).
- ⑤ The final state gives the answer.

## 3. APPROACHES

- **Memoization** (Top-Down)
  - Use recursion.
  - Store results in a table (cache).
  - Compute only needed states.
- **Tabulation** (Bottom-Up)
  - Use iteration.
  - Build table from base cases up to the final state.
  - Often more space-efficient and faster (no recursion overhead).

## 4. GENERAL FORM

$$\begin{aligned} dp[state] &= \min/\max ( \text{cost}(\text{state}, \text{choice}) + dp[\text{next state}] ) \\ \text{Answer} &= dp[\text{target state}] \end{aligned}$$

} min or max depends on the problem (minimization or maximization).

## 5. EXAMPLE - FIBONACCI NUMBERS

Find  $n^{\text{th}}$  Fibonacci number.

**State:**  $dp[n] = n^{\text{th}}$  Fibonacci number

**Recurrence:**  $dp[n] = dp[n-1] + dp[n-2]$  for  $n > 1$

**Base cases:**  $dp[0] = 0, dp[1] = 1$

**Answer:**  $dp[n]$

**Tabulation (Bottom-Up)**

| n     | 0 | 1 | 2 | 3 | 4 | 5 | ... | n     |
|-------|---|---|---|---|---|---|-----|-------|
| dp[n] | 0 | 1 | 1 | 2 | 3 | 5 | ... | dp[n] |

**Time:**  $O(n)$

**Space:**  $O(n)$  (can be optimized to  $O(1)$ )

## 6. TIPS

- Identify overlapping subproblems.
- Define state clearly.
- Handle base cases carefully.
- Choose memoization for sparse states.
- Choose tabulation for iterative solutions and better performance.

Practice makes perfect!

